

Homework 4: Your no-slop setup

Due: Friday, October 9, at 1:10 PM

Submit: one pull request per team to the class repo. One person pastes the link into Courseworks.

Templates and examples: in the `resources/` folder of the class repo.

This week you finish the setup work, so that next week you can start building. Next week you put your landing page together, start bringing people to it, and start the app. Everything here is context you will hand to an AI, so it writes copy and code that fit your product instead of generic slop.

Split the work. Give each deliverable an owner. Research comes first, because the personas and the brand position depend on it. The style guide and the repo setup can start now.

The deliverables

Section 1

1. **Synthesis comparison:** your synthesis, the AI's synthesis, and a short comparison.
2. **Reddit research:** voice of the customer research on Reddit.
3. **Persona agents:** 3 to 5 agents, each playing one of your key users.
4. **Brand position:** polished from the draft you started in class.
5. **Style guide and logo:** colors, type, look and feel, and a set of logo files.
6. **Repo setup:** a feature development framework picked, and a repo ready for coding agents.

Team contracts are separate from this homework. The TAs will send them out for signatures.

1. Synthesis comparison

Section 2

What it is: your read on the interviews, the AI's read on the same interviews, and what the difference taught you.

How to do it:

1. **Part A, your synthesis.** Do this first, before you open an AI. Your synthesis work in class on October 2 is your head start. Write up:
 - the key patterns across interviews
 - your top 5 quotes, in their exact words
 - contradictions, and what they might mean
 - surprises: what changed your thinking
 - your overall read on the data

2. **Part B, the AI's synthesis.** Give an AI your interview snapshots and raw notes. Ask it to synthesize them with the prompt below. Save its full output.

You are a UX researcher. The files I'm attaching are notes and snapshots from [number] user interviews about [the problem you are exploring]. Synthesize them the way a UX researcher would, using thematic analysis.

1. Read every interview before you name any theme.
2. Patterns: the themes that come up across interviews. For each, say which interviews it appears in and how strongly people felt. Strong feeling from one person can matter more than a passing mention from three.
3. Quotes: the 5 most telling quotes. Copy them word for word, and say which interview each one came from. Do not paraphrase.
4. Contradictions: where interviewees disagreed. Do not resolve them or pick a winner. Say what might explain each one, such as two different kinds of user.
5. Surprises: anything that does not fit the patterns.
6. Segments: if the people fall into different groups, describe each group.

Keep separate ideas separate. Do not merge two things different people said into one finding. If the notes do not support a claim, say so instead of guessing.

3. **Part C, the comparison.** One or two paragraphs. What did the AI find that you missed? What did you notice that it didn't? Check every quote the AI gives you against your notes. AIs invent quotes and merge separate ideas into one.

Hand in: `synthesis_comparison.md` , with Parts A, B, and C.

2. Reddit research

Section 3

What it is: The interviews you did were great for high fidelity info transfer (you can read body language and emotion, etc). But not everyone will be honest to your face, and it's

possible for you to lead your interviewees. Reddit shows you what many more people say to each other about the same problem, in their own words without your prompting.

How to do it:

1. Install the Reddit research agent from the class 4 folder of the class repo (`classes/20261002_c4w4_fri_synthesis-and-brand-position/exercise/reddit-research/`). The README explains how.
2. Tell the agent your problem space and your target user. Reddit blocks AI agents, so the agent never loads Reddit itself. You fetch, and the agent reads.
3. **Round 1, searches.** The agent gives you a list of search links ending in `.json` , one per subreddit and topic. Open each in your own browser and save the page into `reddit_corpus/raw/` , keeping the `.json` name. YOU MUST DO THIS BY HAND. Do not automate the opening of links, or you will get rate limited. You also must be logged in to an account. If you get rate limited, wait 10 min and resume.
4. **Round 2, threads.** The agent reads your saved searches and picks the best threads. It gives you a second list of links, this time to the threads themselves. Open and save those too. The threads are where the real comments are.
5. **More rounds if needed.** The agent reads the threads and may ask for a few more. You should end up with 30-100 saved files. Open links at a normal pace; opening dozens at once gets you rate limited and the saved pages come back empty.
6. The agent writes a summary with quotes, each with its link.
7. Pick 3 to 5 quotes for your brand position, and 1-2 quotes per persona.

Hand in: a `reddit_research/` folder with the list of threads you saved, the agent's summary, and the quotes you picked. Each quote needs its exact words, its link, and where you used it.

3. Persona agents

Section 4

What it is: an AI agent that plays one of your key personas. You show it your copy, your page, or a feature idea, and it tells you what is unclear, what is a dealbreaker, and why. These become automated components you can refer to each semester.

How to do it:

1. Find your 3 to 5 key audiences. Your synthesis segments and your Reddit research should give you enough to create these. (Remember: contradictions in your interviews often mean two audiences.)
2. Write one agent file for each, using the persona agent template, `resources/template_persona-agent.md` . `resources/example_persona-agent_maya.md` is a finished example, Maya, from Ken's product Steady Thread. Make each one specific: a name, an age, a situation, what they care about, what they fear, and what makes them say no. Ground every detail in a real quote from your interviews or Reddit.
3. Install the files where your coding tool finds agents. In Claude Code that is `.claude/agents/` .
4. Test each one. Show it your brand position and ask for its reaction. If all your personas say the same thing, they are not specific enough.

Hand in: a `personas/` folder with each agent file, and one short note per persona on what it said about your brand position.

4. Brand position

Section 5

What it is: an internal document your whole team agrees on. It says what problem you solve, who you solve it for, and the exact words you use. Every AI context you open reads it.

How to do it:

1. Start from the draft you made in class.
2. Fill in the brand position template, `resources/template_brand-position.md`. The most important parts are the brand thesis, the positioning statement, the target identity, canonical language, and language to avoid.
3. Use your interviewees' and Reddit users' actual phrases for canonical language.
4. Show it to your persona agents. Fix what they flag (use your judgment).

Hand in: `brand_position.md`.

5. Style guide and logo

Section 6

What it is: the brand position for visuals. Without one, every AI-built page picks its own colors and fonts, and your product looks like slop.

How to do it: you don't fill in a style guide from a blank page. You find it by looking at options, reacting, and narrowing down. Have the AI build the options as HTML pages you open in your browser, and give it feedback in plain words.

The look, in rounds:

1. **Collect references.** 3 to 5 sites or apps whose look you like. Write down what you like about each.
2. **Go wide.** Give the AI your brand position and references. Ask for 5 to 7 sample pages that are very different from each other: a hero and one section each, each a different direction. Different colors, fonts, density, and mood. One trick to use here is to tell the AI "Generate a seed (hash the current datetime) and use that seed as inspiration for each of the different samples in whatever way you would like."
3. **React.** For each option, say what you keep and what you kill, and why. "Keep the type on B, the color is too loud on D, C feels corporate." Write it down; your notes become the guide.

4. **Repeat.** The AI builds a new round that combines what you kept. Reduce the number of samples each round. Eventually ask your team about the top 2-3. (or keep 2-3 and test them later)
5. **Optional: polish the parts.** If you want to go further, vary one thing at a time on your chosen direction: color options, type pairings, buttons and cards, photos or illustrations.
6. **Check it on a real page.** Decisions look different at full size than in a thumbnail. Look at it on your phone, too.
7. **Write it down.** Fill in the style guide template, `resources/template_style-guide.md`, with what you decided. Ask the AI to draft it from your notes, then check it. `resources/example_style-guide_stclair-ai.md` is a finished example.

The logo, in rounds:

1. **Go wide.** Ask the AI for 10 to 20 rough marks on one page, from very different ideas: letters, symbols, and ideas from your brand thesis. Simple shapes work best, drawn as SVG. Sometimes Claude and Codex can do this. Other times, you will want to ask your agent to generate a prompt and give that to Gemini/Nano Banana to generate the image.
2. **Keep and kill.** Say what you keep and why. Run a few rounds until one idea is clearly the direction.
3. **Tune it.** AI is pretty inconsistent at generating the exact same image. If you have the skills and software, tune it by hand in Adobe Illustrator. Otherwise, keep generating until you get one you like.
4. **Check it in place.** Put the finalists next to your product name in a real page header, and at favicon size (16 pixels). Some shapes fall apart small.

5. **Make the full set.** Tell the AI to generate a matrix of your brand mark and wordmark in every version you will need, so the logo works in any setting. The St. Clair AI set, for example, has:

- the mark alone, and a square version
- the lockup: the mark beside the name, with the mark as tall as the name's capital letters
- each of those on a dark background
- one-color versions, in black and in white
- a stacked version and a square avatar for social profiles
- favicons at 16, 32, 48, 180, 192, and 512 pixels

Record the rules in the Logo section of your style guide: how the mark sits next to the name, its colors on light and dark, and the smallest size it works at.

Hand in: `style_guide.md`, and a `logo/` folder with all your logo files.

6. Repo setup

Section 7

What it is: the coding half of the setup. A repo a coding agent can walk into and understand.

How to do it:

1. **Pick a feature development framework.** Go through the frameworks in the table below and pick one. These are processes for building features with coding agents: how you plan, build, review, and ship. Write down which one you picked and why.
2. **Create your product repo.**
3. **Write AGENTS.md** at the top of the repo, using the template, `resources/template_agents-md.md`. It tells coding agents how your team works, how to run the project, and where the docs are.
4. **Add a docs/ folder** with your brand position and style guide.

Hand in: `repo_setup.md` , with one line on the framework you chose and why, and a copy of your AGENTS.md. Your product repo does not have to be public. If it is, add a link.

Feature development frameworks

All of these were updated in the last few weeks. Last updated is the most recent change on GitHub, checked October 2, 2026. pstack now lives off GitHub; its date is for the GitHub mirror.

Framework	Repo	What it is	Last updated
Feature Forge	github.com/kenxle/feature-forge	Ken's process: brief, plan, build, verify, ship, learn.	2026-09-30
gstack	github.com/garrytan/gstack	Garry Tan's Claude Code setup: role-based tools for planning, design, review, QA, and release.	2026-10-02
Superpowers	github.com/obra/superpowers	A skills library and development method: brainstorm, plan, test-first, review.	2026-09-27
pstack	x.ai/bot/plugin/9717366 · mirror: github.com/backnotprop/pstack	Skills and principles for rigorous AI-assisted engineering, from a GrokBot engineer.	2026-09-14 (mirror)
GitHub Spec Kit	github.com/github/spec-kit	Spec-driven development: write the spec, then plan and build from it.	2026-10-02
OpenSpec	github.com/Fission-AI/OpenSpec	Lightweight spec-driven development	2026-10-02

Framework	Repo	What it is	Last updated
		for AI coding assistants.	
BMAD Method	github.com/bmad-code-org/BMAD-METHOD	Agile AI-driven development: clarify, plan, build, and learn, with product, architecture, and testing perspectives.	2026-10-02
Compound Engineering	github.com/EveryInc/compound-engineering-plugin	Every's plugin: brainstorm, plan, build, review, then capture what you learned.	2026-10-02
Agent OS	github.com/buildermethods/agent-os	Captures your codebase standards and writes better specs for agents.	2026-08-29
SuperClaude	github.com/SuperClaude-Org/SuperClaude_Framework	Commands, personas, and methods layered on Claude Code.	2026-09-27
claude-flow (Ruflo)	github.com/ruvnet/ruflo	Multi-agent swarms and orchestration. Powerful, and the most complex here.	2026-10-02

Folder layout

Section 8

```
teams/your-team-name/hw4-synthesis-brand-position/  
  synthesis_comparison.md  
  reddit_research/  
  personas/  
  brand_position.md  
  style_guide.md  
  logo/  
  repo_setup.md
```